

Evaluating Circuits

Pedro Zuidberg Dos Martires

DPLL with Component-Caching #SAT

Algorithm 1: Component-Caching #SAT

```
if  $S = \emptyset$  then  
  | return  $2^{|Vars|}$   
if  $S$  contains an empty clause then  
  | return 0  
if  $S$  is in cache then  
  | return cache[ $S$ ]  
select  $v \in Vars$   
 $S_t \leftarrow S[v = \top]$   
 $S_f \leftarrow S[v = \perp]$   
 $r \leftarrow \#SAT(Vars \setminus \{v\}, S_t) + \#SAT(Vars \setminus \{v\}, S_f)$   
Cache[ $S$ ]  $\leftarrow r$   
return  $r$ 
```

DPLL Variant with Decomposition for #SAT

Algorithm 2: Component-Caching #SAT with Decomposition

```
if  $S = \emptyset$  then
  return  $2^{|Vars|}$ 
if  $S$  contains an empty clause then
  return 0
if  $S$  is in cache then
  return Cache[ $S$ ]
decompose  $S$  into independent components  $S_1, \dots, S_k$ 
if  $k > 1$  then
   $r \leftarrow \prod_{i=1}^k \text{\#SAT}(Vars_i, S_i)$ 
  Cache[ $S$ ]  $\leftarrow r$ 
  return  $r$ 
select  $v \in Vars$ 
 $S_t \leftarrow S[v = \top]$ 
 $S_f \leftarrow S[v = \perp]$ 
 $r \leftarrow \text{\#SAT}(Vars \setminus \{v\}, S_t) + \text{\#SAT}(Vars \setminus \{v\}, S_f)$ 
Cache[ $S$ ]  $\leftarrow r$ 
return  $r$ 
```

The Same Search, Different Computations

Let $\phi = \phi_1 \wedge \phi_2$ denote independent components.

$$\text{SAT}(\phi) = \text{SAT}(\phi_{v=1}) \vee \text{SAT}(\phi_{v=0})$$

$$\text{SAT}(\phi) = \text{SAT}(\phi_1) \wedge \text{SAT}(\phi_2)$$

$$\#\text{SAT}(\phi) = \#\text{SAT}(\phi_{v=1}) + \#\text{SAT}(\phi_{v=0})$$

$$\#\text{SAT}(\phi) = \#\text{SAT}(\phi_1) \times \#\text{SAT}(\phi_2)$$

$$\text{WMC}(\phi) = w(v) \text{WMC}(\phi_{v=1}) + w(\neg v) \text{WMC}(\phi_{v=0})$$

$$\text{WMC}(\phi) = \text{WMC}(\phi_1) \times \text{WMC}(\phi_2)$$

A Common Pattern

Every recursive solver performs two operations.

Branching

$$f(\phi) = f(\phi_{v=1}) \oplus f(\phi_{v=0})$$

Decomposition

$$f(\phi) = f(\phi_1) \otimes f(\phi_2)$$

when ϕ_1 and ϕ_2 have disjoint variables.

Different inference tasks only change $\oplus$ and $\otimes$.

Algebraic Model Counting

Choose

$$(\oplus, \otimes)$$

from some algebraic structure.

Then evaluate the same search procedure:

$$f(\phi) = f(\phi_{v=1}) \oplus f(\phi_{v=0})$$

$$f(\phi) = f(\phi_1) \otimes f(\phi_2)$$

Examples of Algebraic Model Counting

$$(\vee, \wedge) \rightarrow SAT$$

$$(+, \times) \rightarrow \#SAT$$

$$(+, \times) \rightarrow WMC$$

Algebraic Model Counting generalizes all three.

Most Probable Explanation (MPE)

Suppose we observe some evidence e .

Instead of computing a probability, we ask:

$$\begin{aligned}\text{MPE}(e) &= \max_x p(x \mid e) \\ &= \max_x \frac{p(x, e)}{p(e)} \\ &= \max_x p(x, e)\end{aligned}$$

Question:

Which possible world is the most likely?

Most Probable Explanation (MPE)

Weighted Model Counting:

$$p(e) = \sum_x p(x, e)$$

Most Probable Explanation:

$$\text{MPE}(e) = \max_x p(x, e)$$

Replace summation by maximization.

MPE as a Recursive Computation

For Weighted Model Counting:

$$WMC(\phi) = w(v) WMC(\phi_{v=1}) + w(\neg v) WMC(\phi_{v=0})$$

For Most Probable Explanation:

$$MPE(\phi) = \max \left(w(v) MPE(\phi_{v=1}), w(\neg v) MPE(\phi_{v=0}) \right)$$

Independent components still multiply:

$$MPE(\phi_1 \wedge \phi_2) = MPE(\phi_1) \cdot MPE(\phi_2)$$

Examples

Task	$\oplus$	$\otimes$
SAT	$\vee$	$\wedge$
#SAT	$+$	$\times$
WMC	$+$	$\times$
MPE	max	$\times$

Same search procedure.
Different algebra.

Semirings

A semiring consists of

$$(\mathcal{A}, \oplus, \otimes, e^{\oplus}, e^{\otimes})$$

where:

- ▶ $\mathcal{A}$ is the set of values
- ▶ $\oplus$ combines alternative solutions
- ▶ $\otimes$ combines independent subproblems
- ▶ $e^{\oplus}$ is the neutral element for $\oplus$
- ▶ $e^{\otimes}$ is the neutral element for $\otimes$

$$a \oplus e^{\oplus} = a \quad a \otimes e^{\otimes} = a$$

The algebra determines the inference task.

Semiring Properties

A semiring $(\mathcal{A}, \oplus, \otimes, e^{\oplus}, e^{\otimes})$ satisfies:

Associativity

$$(a \oplus b) \oplus c = a \oplus (b \oplus c)$$

$$(a \otimes b) \otimes c = a \otimes (b \otimes c)$$

Commutativity

$$a \oplus b = b \oplus a$$

$$a \otimes b = b \otimes a$$

Distributivity

$$a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$$

Neutral Elements

$$a \oplus e^{\oplus} = a \quad a \otimes e^{\otimes} = a$$

Why Do We Need These Properties?

Associativity allows us to regroup computations.

Commutativity allows us to reorder computations.

Distributivity allows local computations to represent exponentially many worlds.

Neutral elements define the base cases of the recursion.

Examples of Semirings

Task	$\mathcal{A}$	$\oplus$	$\otimes$	$e^\oplus$	$e^\otimes$
SAT	$\{\perp, \top\}$	$\vee$	$\wedge$	$\perp$	$\top$
#SAT	$\mathbb{N}$	$+$	$\times$	0	1
WMC	$\mathbb{R}_{\geq 0}$	$+$	$\times$	0	1
MPE	$\mathbb{R}_{\geq 0}$	max	$\times$	0	1
Log-MPE	$\mathbb{R} \cup \{-\infty\}$	max	$+$	$-\infty$	0

Same search procedure. Different semirings.

Everything is Counting

Suppose we recursively explore all possible worlds.

What changes from one task to another is not the search space.

What changes is how we aggregate the worlds.

SAT	:	does at least one world exist?
#SAT	:	how many worlds exist?
WMC	:	what is the total weight of the worlds?
MPE	:	what is the weight of the best world?

Inference = Counting = Aggregation

Pointers to the Literature

Angelika Kimmig, Guy Van den Broeck, and Luc De Raedt (2017). “Algebraic model counting”. In: Journal of Applied Logic 22, pp. 46–62

Rafael Kiesel, Pietro Totis, and Angelika Kimmig (2022). “Efficient knowledge compilation beyond weighted model counting”. In: Theory and Practice of Logic Programming 22.4, pp. 505–522

Benjie Wang et al. (2024). “A Compositional Atlas for Algebraic Circuits”. In: Advances in Neural Information Processing Systems

Pedro Zuidberg Dos Martires, Luc De Raedt, and Angelika Kimmig (2024). “Declarative probabilistic logic programming in discrete-continuous domains”. In: Artificial Intelligence 337, p. 104227

Lennert De Smet and Luc De Raedt (2025). “Defining neurosymbolic ai”. In: arXiv preprint arXiv:2507.11127

Why Do We Care?

The search structure is independent of the task.



Once the circuit has been constructed, we can evaluate it using different semirings.

$(\vee, \wedge)$ $(+, \times)$ $(\max, \times)$

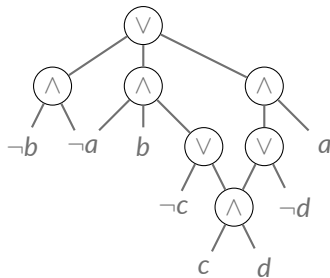
Compile once. Evaluate many times.

Circuit Example

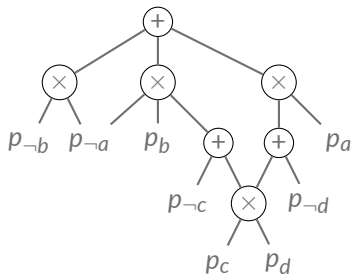
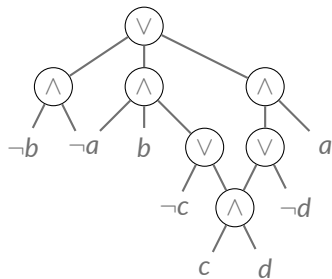
Consider the formula

$$\phi = D \vee (A \wedge C) \vee (\neg A \wedge \neg B) \vee (\neg A \wedge \neg C).$$

Using a knowledge compiler we get:



Changing the Algebra



Probabilistic Inference as Circuit Evaluation

Algorithm 3: Eval(n)

```
if  $n$  is in cache then
   $\lfloor$  return cache[ $n$ ]
if  $n$  is a literal  $\ell$  then
   $\lfloor$   $r \leftarrow w(\ell)$ 
else if  $n$  is a  $\times$ -node then
   $\lfloor$   $r \leftarrow \prod_{c \in \text{children}(n)} \text{Eval}(c)$ 
else if  $n$  is a  $+$ -node then
   $\lfloor$   $r \leftarrow \sum_{c \in \text{children}(n)} \text{Eval}(c)$ 
cache[ $n$ ]  $\leftarrow r$ 
return  $r$ 
```

From Search to Circuits

DPLL explores a search space.

Caching turns the search tree into a DAG.

Decomposition introduces product nodes.

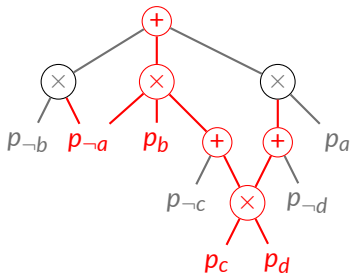
Search Tree $\longrightarrow$ Search DAG $\longrightarrow$ Compiled Circuit

Compilation is expensive. Evaluation is easy.

Compile once. Evaluate often.

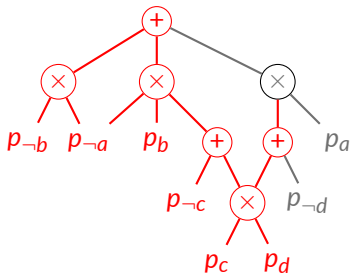
Computing Probabilities

We want to get the probability $p(a = 0, b = 1, c = 1, d = 1)$



Computing Marginals

We want to get the probability $p(a = 0, d = 1)$



Marginals as Indicator-Weighted Evaluation

To compute a marginal query $p(q)$, we evaluate the circuit with modified leaf values.

For every literal ℓ :

$$\lambda_q(\ell) = w(\ell) \cdot \mathbb{I}[q \models \ell]$$

where

$$\mathbb{I}[q \models \ell] = \begin{cases} 1 & \text{if } q \text{ is compatible with } \ell \\ 0 & \text{otherwise.} \end{cases}$$

Then

$$p(q) = \text{Eval}(\text{root})$$

The indicators select worlds compatible with the query.

Marginal Inference

Algorithm 4: Marginal Inference by Indicator-Weighted Evaluation

Input: Circuit root r , query assignment q

Output: Marginal probability $p(q)$

foreach literal ℓ in the circuit **do**

$\lambda_q(\ell) \leftarrow w(\ell) \cdot \mathbb{I}[q \models \ell]$

return Eval(r)

$$p(q) = \sum_{x \models q} \prod_{\ell \in x} w(\ell)$$

The Benefit of Compilation

Without a circuit:

$$p(q) = \sum_{x \models q} \prod_{\ell \in x} w(\ell)$$

requires summing over exponentially many worlds.

With a compiled circuit:

$$p(q) = \text{Eval}(\mathcal{C}, q)$$

requires a single pass through the DAG.

Compilation turns marginalization into circuit evaluation.

A Challenge for Neurosymbolic AI

Modern neural networks are efficient because they can be expressed as

matrix-vector multiplications

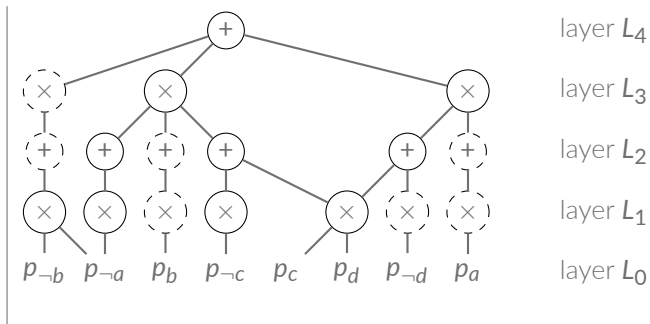
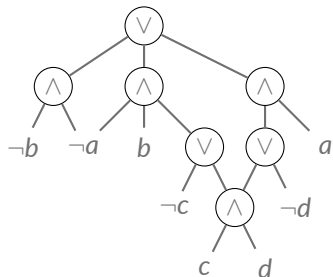
which execute extremely well on GPUs.

Compiled circuits look very different. They are

- ▶ sparse,
- ▶ irregular,
- ▶ highly heterogeneous,
- ▶ full of branching structure.

Can we map circuits to more regular computation graphs?

How? Use Knowledge Layers (KLayer)¹



¹Maene, Derkinderen, and Zuidberg Dos Martires 2025.

for each node n in a circuit C we compute its height in the circuit h_n
nodes with the same height are assigned to the same layer.

$$L_i = \{n \in C \mid h_n = i\} \quad \text{where} \quad h_n = \begin{cases} 0, & \text{if } n \text{ is a leaf,} \\ \max_{c \in \mathcal{C}_n} h_c + 1 & \text{otherwise.} \end{cases} \quad (1)$$

$\mathcal{C}_n$ denotes the set of children of a node n

for each node n in a circuit C we compute its height in the circuit h_n
nodes with the same height are assigned to the same layer.

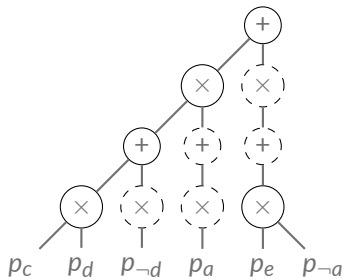
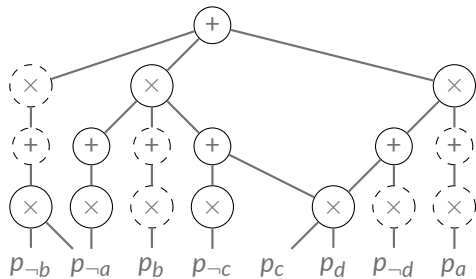
$$L_i = \{n \in C \mid h_n = i\} \quad \text{where} \quad h_n = \begin{cases} 0, & \text{if } n \text{ is a leaf,} \\ \max_{c \in C_n} h_c + 1 & \text{otherwise.} \end{cases} \quad (1)$$

C_n denotes the set of children of a node n

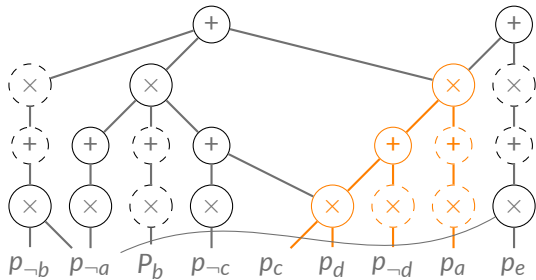
We guarantee to have the same type of computation in each layer by introducing dummy units.

We force layers to only depend on immediately preceding layers. No “skip connections”.

Merging circuits ...



...into multi-rooted circuits.



Sub-Circuit Detection

Theorem

Given a set of circuits, all identical sub-circuits can be identified in linear expected time complexity in the total number of edges in the circuits.

Sub-Circuit Detection

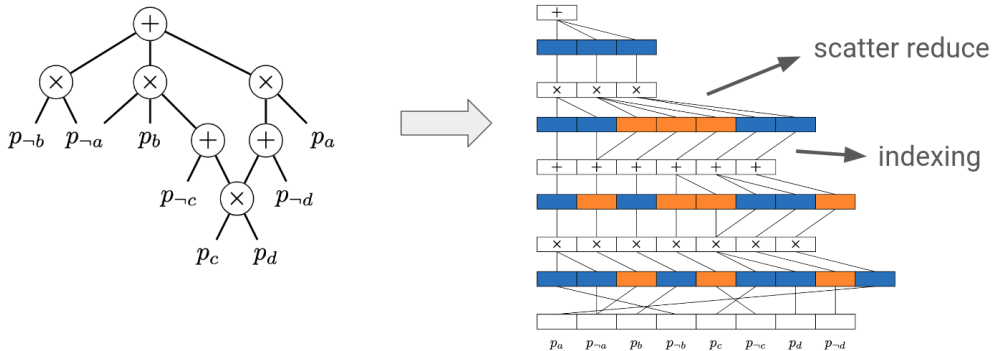
Theorem

Given a set of circuits, all identical sub-circuits can be identified in linear expected time complexity in the total number of edges in the circuits.

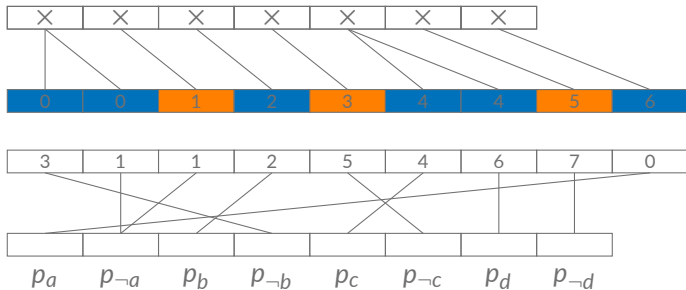
Syntactic equivalence in general and semantic equivalence for canonical circuits.

In practice we performed function comparison using Merkle hashes (c.f. git, blockchain).

KLay reduces circuit evaluations to scatter-reduce operations



Layer Evaluation as a Segmentatoin



node values N_1

aggregation indices R_1

edge indices S_1

node values N_0

Layer Evaluation Using GNN Primitives

```
from jax.ops import segment_sum, segment_prod
```

```
def sum_layer(num_segments, ix_in, ix_out, x):  
    return segment_sum(  
        x[ix_in], ix_out,  
        num_segments=num_segments,  
        indices_are_sorted=True  
    )
```

```
def prod_layer(num_segments, ix_in, ix_out, x):  
    return segment_prod(  
        x[ix_in], ix_out,  
        num_segments=num_segments,  
        indices_are_sorted=True  
    )
```

The Circuit Evaluation Algorithm

Algorithm 5: KLayout Forward Evaluation

Input: N_0 , selection indices $S_1, S_2, \dots, S_L$, reduction indices $R_1, R_2, \dots, R_L$

for $l \leftarrow 1$ to L do

$E_l \leftarrow N_{l-1}[S_l];$

 if $l \bmod 2 = 0$ then

$N_l \leftarrow \text{scatter}(E_l, R_l, \text{reduce}=\text{'sum'});$

 else

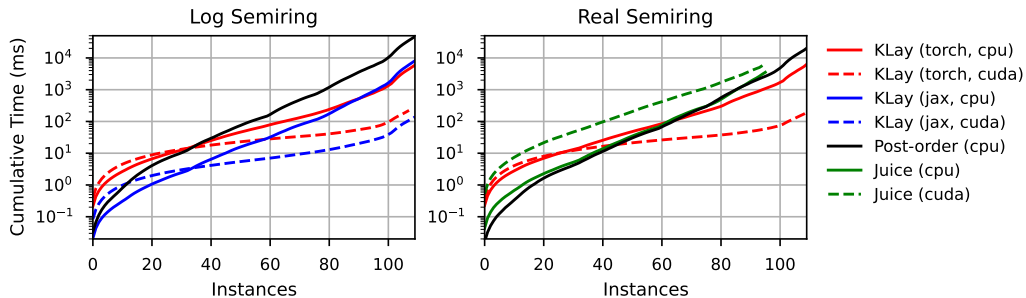
$N_l \leftarrow \text{scatter}(E_l, R_l, \text{reduce}=\text{'product'});$

return N_L ;

General Workflow

1. Read in a compiled circuit from any knowledge compiler
2. Layerize the circuit
3. Interpret the circuit as a layered computation graph
4. Just-in-time compile the computation graph

Experimental Results



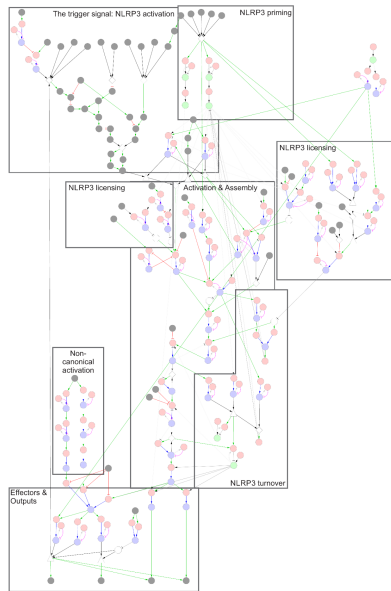
Experimental Results

	Sudoku (4×4)	Grid (4×4)	HMLC	Warcraft (12×12)
SDD Nodes	2 408	6 610	9 730	1 110 234
KLay Nodes	3 926	6 590	26 306	1 155 063
KLay Layers	30	24	84	84
Naive (cpu)	17.35 $\pm$ 0.05	46.12 $\pm$ 0.49	127.00 $\pm$ 140.03	11484.53 $\pm$ 251.94
Naive (cuda)	42.70 $\pm$ 2.76	107.78 $\pm$ 4.67	175.58 $\pm$ 1.26	18930.67 $\pm$ 496.16
KLay (cpu)	0.45 $\pm$ 0.02	0.53 $\pm$ 0.03	1.41 $\pm$ 0.02	18.55 $\pm$ 0.14
KLay (cuda)	0.46 $\pm$ 0.17	0.49 $\pm$ 0.19	1.03 $\pm$ 0.35	1.59 $\pm$ 0.25
Neural (cpu)	0.90 $\pm$ 0.63	0.17 $\pm$ 0.02	0.55 $\pm$ 0.29	242.60 $\pm$ 4.81
Neural (cuda)	0.51 $\pm$ 0.13	0.34 $\pm$ 0.12	0.20 $\pm$ 0.04	2.80 $\pm$ 0.07

Sidenote

We developed knowledge layers out of necessity because of the scaling issues when modelling biological signalling transduction networks probabilistically^a.

^aPedro Zuidberg Dos Martires, Vincent Derkinderen, et al. (2024). "Automated Reasoning in Systems Biology: a Necessity for Precision Medicine". In: Proceedings of the International Conference on Knowledge Representation and Reasoning.



Circuits as Structured Computation Graphs

So far we have assumed that

$$p(q) = \text{Eval}(\mathcal{C}, q)$$

can be computed correctly and efficiently.

Why?

Because compiled circuits satisfy important structural properties:

- ▶ Determinism
- ▶ Decomposability
- ▶ Smoothness

Determinism

A sum node is deterministic if at most one child can be satisfied.

$$c_i \wedge c_j \equiv \perp$$

for all $i \neq j$.

$(A \wedge B) \vee (\neg A \wedge C)$ is deterministic.

$(A \wedge B) \vee (A \wedge C)$ is not.

Why Determinism Matters

Recall:

$$V \rightarrow +$$

This is only correct when the children are mutually exclusive.

If

$$A \wedge B = \perp$$

then

$$P(A \vee B) = P(A) + P(B)$$

Decomposability

A product node is decomposable if its children have disjoint variable sets.

$$\text{Vars}(c_i) \cap \text{Vars}(c_j) = \emptyset$$

for all $i \neq j$.

Why Decomposability Matters

Recall the decomposition step:

$$\phi = \phi_1 \wedge \phi_2$$

where

$$\text{Vars}(\phi_1) \cap \text{Vars}(\phi_2) = \emptyset.$$

Then

$$\text{WMC}(\phi) = \text{WMC}(\phi_1) \cdot \text{WMC}(\phi_2).$$

Decomposability allows local products.

Smoothness

A sum node is smooth if all children mention the same variables.

Smoothness is not required for basic weighted model counting.

However, smoothness simplifies:

- ▶ probabilistic semantics,
- ▶ marginal computation,
- ▶ normalization.

Missing variables can often be implicitly completed because

$$w(x) + w(\neg x) = 1.$$

Circuit Properties

Property	Why We Need It
Determinism	Prevent double counting
Decomposability	Valid products
Smoothness	Make sure all variables appear

Determinism + Decomposability + Smoothness

Structure enables tractable inference.